

L8 总结：路由 / ReAct / Plan-and-Execute

JD 6/10。本节封版日 2026-07-27（跨两次 session：Windows 起头写讲义+quiz+Part A 一半，macOS 续完作业）。本节的独特之处：几乎不教新代码结构，而是给你 L2 就写出来的东西正式命名、划清边界，再用一场亲手跑的对比实验把权衡从"我告诉你的"变成"你自己撞出来的"。

一、一句话抓住本节

你 L2 手写的 `while True agent loop` 本质就是 **ReAct** (Reason→Act→Observe)。L8 干三件事：1. **正名**：给这个"涌现的循环"装上术语 (ReAct) 和边界。2. **升级到生产级**：把玩具 `while True` 改造成有**显式终止语义**、**纯逻辑可测**、**结构化日志**的 `executor`。3. **实测两范式**：ReAct vs Plan-and-Execute 在三类任务上真跑，用数据回答"什么时候该显式规划"。

三个必背锚点（其余表格全可现场推导）：- **锚点①**：我的 L2 loop = ReAct；原版靠正则解析文本，现代版靠结构化 `tool_calls` 字段。- **锚点②**：一条轴 = **重规划频率**。ReAct（每步重规划）↔ Plan-and-Execute（只规划一次），真实系统落中间。- **锚点③**：**自主性是成本，不是收益**。只在"环境不可预测 + 试错便宜"两条同时成立时才买它（coding agent 满足 → 纯 ReAct；客服 agent 不满足 → 静态路由 + 分支内受限短 loop）。

二、作业实战：把玩具 loop 升级成生产件

作业分三部分共 25 个 TODO，全部完成；`test_executor.py` 11 个测试全绿。

Part A · `executor.py` (ReAct executor, 7 TODO)

设计红线：纯逻辑 / 副作用分离（回扣 L6 v2 作业）—— - `LoopGuard`：纯逻辑，**不碰网络**，只回答"该不该停、为什么停"。这是 `pytest` 的靶子。- `run_react`：接线层，碰 API 和工具，判断全部委托给 `LoopGuard` 和 `_execute_tool`。

几个亲手拍板的设计决策（面试都会问）：

- 终止语义显式枚举**，不返回字符串。`StopReason` 六个成员分两类：正常（`FINAL_ANSWER` / `HANDOFF`）+ 被迫（`MAX_STEPS` / `NO_PROGRESS` / `BUDGET_EXCEEDED` / `TOOL_FATAL`）。为什么必须枚举 → **可观测**（L13 `trace` 字段）、**可测试**（`pytest` 断言它）、**上层能分情况兜底**。本节还新增了两个：`INVALID_PLAN`（计划有环/幻觉依赖）、`REPLAN_EXHAUSTED`（重规划额度耗尽）。
- `should_stop` 的检查顺序是设计，不是随便排**：`budget` 排在 `max_steps` 前，理由——**异常信号（钱烧光）比常态（步数走完）更该被 `trace` 看见**。
- 边界用 `>=` 不用 `>`**：`should_stop` 在 `record_step` 之后调用，`_steps` 已含刚走完那步。用 `>` 会多放行一轮 = **多一次真实 API 调用（多花钱 + 多延迟）**，且上限对不上账（说限 8 步实跑 9 步，面试穿帮）。
- `no-progress` 检测的陷阱**：连续 N 轮指纹相同才算转圈。但空指纹 `[]`（模型没点工具）不能算——加 `first and` 排掉空基准。纯逻辑不能依赖调用方不喂 `[]`（单测会直接喂）。
- `_execute_tool` 三类错误分流**（本节最精的一段，L3 的落地）：
 - `BusinessError` → 原文回传，模型能自愈，`fatal=False`
 - `FatalError` → 内部记详细日志、外部只回脱敏话术"服务暂时不可用"，`fatal=True`

- 8. `TransientError` → 本节按 fatal 处理，留 `# TODO(L13)`：重试
- 9. 硬约束：任何路径都必须返回字符串（每个 `tool_call_id` 必须有且仅有一条回复），异常绝不能逃出去。
- 10. 生产级升级：调用前用 `inspect.signature().bind()` 校验参数，把"模型传错参数名"（可自愈）和"工具内部 bug"（该告警）分开——否则我们自己的 bug 会被当成模型的锅无限重试。
- 11. fatal 后不再触发副作用（决策 2(B)）：一个工具 fatal 后，本轮剩余工具跳过执行、只补"已取消"消息。验证过：数据库挂了时 `apply_compensation`（不可逆写操作）一次没执行，赔付记录为空。

Part B · `planner.py`（Plan-and-Execute + replan, 5 TODO）

顺手补上了 L6 欠的 Part B（结构化输出）——让模型用 schema 吐"计划对象"，而非要正则解析的自然语言。

- `PLAN_SCHEMA`： `arguments` 选了存JSON 字符串而非自由 object——绕开 strict 模式对自由 object 的拒绝， 且和 `_execute_tool` 的 `str` 入参天然对齐。新增 `worth_planning` 字段让模型能说"这任务不值得规划"。
- `parallel_groups`： Kahn 拓扑排序的分层版，同层可并发。三件事：分层算法 + 环检测（抛 `ValueError`）
- 幻觉依赖检测（依赖不存在的 id）。返回 `list[set[str]]`（层内无序，用 set 表达最贴切）。
- `is_plan_stale`：计划过时闸门。只实现了信号①（检测 error）；信号②（观察与计划假设矛盾，如 `delayed=false` 却要赔付）留了注释说明取舍——通用做法是 LLM-as-judge，贵；规则法便宜但不通用。
- `make_plan`：撞上 DeepSeek 端点 不支持 `json_schema` (400)，降级到 `json_object` + 把字段要求写进 prompt。解析失败兜底成 `worth_planning=False` 的空计划，让上层优雅退化——graceful degradation。规划用慢模型 `deepseek-v4-pro`（规划错一次全盘皆输），执行/答复用 `deepseek-v4-flash`。
- `run_plan_execute`： Plan → Execute → 偏差触发 replan。token 三处累计（初始规划 + 每次 replan + 最终答复），漏一处对比就是假的。

Part C · `compare.py` + `findings.md`（对比实验，3 TODO）

设计了三类任务：`single_step` (T1) / `parallel_fanout` (T2) / `branch_on_observation` (T3)，每个都跑前先押注 `expectation`，跑完看打脸没有。

一个关键的度量修正：最初 `measure()` 用 `result.steps` 当 LLM 调用数，但 P&E 的 `steps` 数的是 工具执行数不是模型调用数——会谎报"P&E 单步省一半"。修法：给 `RunResult` 加独立 `llm_calls` 字段，两个 executor 各自如实计数。教训：对比实验里，度量口径错一个，结论全废。

三、Part C 的三个打脸（本节智力高潮）

真实数据（DeepSeek，2026-07-27）：

任务	模式	LLM调用	tokens	耗时	正确？
T1 单步	ReAct	3	2334	6.44s	✅
T1 单步	P&E	2	864	6.14s	✅ 更精简
T2 扇出	ReAct	4	3956	7.44s	✅ 实赔 A123/C789
T2 扇出	P&E	2	831	29.6s	❌ 幻觉翻车
T3 分支	ReAct	2	1445	3.28s	✅
T3 分支	P&E	3	1912	12.31s	✅ 靠 replan 自救但慢 4 倍

打脸① (T1, 部分): 预期"P&E 白付规划延迟"没成立——P&E 反而 token 更省、延迟基本打平。但打平的原因是它的规划走慢模型, 1 次慢规划 $\approx$ 2 次 flash, 吃掉了"调用数少"的优势。→ 调用数少 $\neq$ 更快, 要看每次调用的成本。

打脸② (T2, 完全) ——最劲爆: 预期"P&E 赢并行", 实际 P&E 幻觉翻车: 答案编出不存在的订单 ID 001/002 和假日期, 一笔赔付都没执行 (日志无 `compensation_applied`)。831 token 看着"最省", 实则是"啥也没干"的副产物——只看数字会把失败读成高效。深层原因: 扇出目标 (订单号) 要先 `list_recent_orders` 才知道, 是运行时才发现的; 而 P&E 规划期就得定死, 此刻不知道有哪些订单 → 没法规划 → 幻觉。

金句: 扇出任务目标"运行时发现" vs P&E 要求"规划期已知"——时序天生冲突。独立可并行还不够, 目标必须规划期已知, P&E 才能并行。

打脸③ (T3, 部分): 预期"P&E 翻车", 实际 P&E 答案也对 (靠 1 次 `replan` 自救: 计划给未延迟的 B456 排赔付 → `amount` 非法 → `BusinessError` → `error` → 信号①命中 → `replan` → 新计划不赔), 但慢 ReAct 4 倍。ReAct 对 `branch-on-observation` 是原生处理 (看到观察再决定), P&E 得"先错→再 `replan`"。

`max_replans=0` 现场 (关掉安全网): `replan_exhausted` | `llm_calls=1` | 赔付记录={} | `answer=None`。失败形态是"侦测到→无力修复→安全放弃 (不出答案)", 不是"乱赔一通"。但这次能安全中止是运气——模型恰好把 `amount` 填成非法值撞上了信号①; 若它猜成一个合法正数, 就会真给未延迟订单赔款且检测不到 (信号②没实现)。那才是真正的"机械执行过时计划且无人察觉"。→ 这是 `is_plan_stale` 只做①的安全网漏洞, 生产要补② (LLM-as-judge)。

大结论: ReAct 三战全胜或打平。P&E 的理论 token 优势全部蒸发 (T1 被慢规划抵消、T2 靠幻觉"省"、T3 靠 `replan` 补救但慢)。客服多为单步/浅分支且目标运行时发现, 故以 ReAct 为主, 仅在目标规划期已知且真可并行的批处理子任务上局部用 P&E。

四、反复出现的短板 (教练视角, 务必继续盯)

头号短板"多部分任务只做一半/改一半"本节又出现多次, 是第 6~7 次: - `record_step` 里 `self.total_prompt_tokens += ...` 漏下划线 (撞只读 property), 同方法内 `self._steps += 1` 却写对了——同类操作两种写法, 典型改一半。- `is_plan_stale` 写完 `try/except` 后漏了末尾 `return False`, 正常观察静默返回 `None` (违约 → `bool`)。- `trace` 声明了、传进 `_result` 了, 但从头到尾没 `append` 过——容器建了没人往里写。- `run_react` 里把我免费给的 `messages.append(_normalize(msg))` 连注释一起删掉了——导致 `assistant` 消息丢失、`messages` 序列非法 (真实 API 会 400)。

已固化的自查动作 (下次继续督促): 1. 写完一个有返回值的函数, 扫一遍每条分支是否都 `return`。2. 声明一个容器, 当场问"谁往里写"。3. 改代码按块删时, 回头确认删掉的都是该删的。4. `commit` 前 `git diff` 扫一眼 (本节 `from attrs import inspect` 覆盖标准库 `inspect`, 自动导入塞进来的, 报错信息完全不提 `import` 冲突, 从错误往回找很费劲——`git diff` 扫 `import` 区能拦住)。

工程直觉在长: 本节多处设计决策 (`stop_reason` 顺序、`>=` 边界、`fatal` 后不触发副作用、度量口径修正) 都能说出为什么, 且主动要求对齐生产规范。

五、quiz Q4 的教学发现 (后续出题可复用)

给一段有问题的 `loop` 找 bug, 他找出的 4 条全是通用工程问题 (`json.loads` 在 `try` 外、无步数上限、`result` 不保证是 `str`、上下文无限增长——相当漂亮), 但本节刚讲的 4 条一个没提 (`stop_reason` 枚举、`fatal` 分流、`no-progress`、`print` 残留)。→ 通用工程直觉在长, 但"刚学的东西还没变成扫代码的检查清单"。后续出题继续用这个手法验证。

六、进 interview-notes 的素材

1. "现代 function-calling agent loop = ReAct 的协议化版本"——你不是没学过 ReAct，是一上来就写的工业版。
 2. "ReAct 和 P&E 不是两种架构，是重规划频率这条轴的两端"——真实系统落中间，故主流是 Plan→Execute→偏差触发 Replan。
 3. "自主性是成本，不是收益"——只在环境不可预测+试错便宜时买。coding agent 满足→纯 ReAct；客服不满足→路由+受限 loop。
 4. "扇出任务目标运行时发现 vs P&E 规划期定死，时序天生冲突"（T2 实测幻觉，王牌回答）。
 5. "光看 token/调用数会把失败读成高效"——P&E 最省那次其实啥也没干，故 correct 列必须人工判（L12 LLM-judge 的动机）。
 6. stop_reason 要如实说出哪道闸门拦住你——计数闸（MAX_STEPS/REPLAN_EXHAUSTED）≠ 金钱闸（BUDGET_EXCEEDED），别乱报。
 7. 工具执行层两级区分：调用前 inspect.signature().bind() 校验参数（模型的锅，回传自愈）；调用后异常按业务/瞬时/致命三类分流（致命错内部详细日志、外部脱敏话术）。
 8. json_schema 不支持时优雅降级 json_object——真实战况，6 次规划全降级，graceful degradation 实例。
-

七、遗留与下一步

- 未做：is_plan_stale 信号②（语义矛盾检测）——留待 capstone，需 LLM-as-judge。
- 未做：worth_planning=False 时回退 ReAct（本版走了空计划，不崩但没利用信号）——capstone 补。
- 未做：API 调用层的超时/限流(429)/重试——归 L13。
- v3 锚点：smolagents CodeAgent（≤90min，第一个锚点，做成模板）——可做可先跳，看真实框架怎么实现"代码即动作"（CodeAct，讲义 §7 预告）。
- 测试系统学习仍排在 capstone；本节测试是"教练搭骨架、他补断言"（10 个 TODO-T 全补齐、11 绿）。